Marine Species Classification

Neural Networks and Deep Learning — Spring 2026

Course Code: CIE 555

1. Overview

This assignment requires students to develop and evaluate deep learning models for the classification of marine organisms using the FathomNet underwater imagery dataset. The dataset encompasses a diverse range of aquatic species and environments. Students are expected to construct convolutional neural networks from first principles, apply transfer learning methodologies, and assess model performance using a comprehensive suite of evaluation metrics.

2. Learning Objectives

Upon successful completion of this assignment, students will be able to:

- Design, implement, and train a convolutional neural network from scratch for multi-class image classification.
- Apply transfer learning techniques, including the use of pretrained backbones and systematic fine-tuning, and critically compare the performance of different strategies.
- Construct and justify preprocessing pipelines appropriate for underwater imagery.
- Produce rigorous quantitative evaluations, informative visualisations, and a concise technical report.

3. Dataset

The FathomNet open-access database of underwater imagery is to be used as the primary dataset for this assignment. It is available at: https://www.fathomnet.org/

4. Mandatory Requirements

The following requirements must be fulfilled and clearly documented in the submitted Jupyter Notebook and accompanying technical report.

4.1 Data Preparation

Students must thoroughly document every stage of the data preparation pipeline, including all design decisions and their rationale.

4.2 Model A — Convolutional Neural Network from Scratch

Students shall design and implement an original convolutional neural network architecture. The following specifications and justifications are required:

- The minimum permissible architecture comprises three convolutional blocks (each consisting of a convolutional layer, an activation function, and a pooling layer), followed by a fully connected classification head.
- A written justification must be provided for all architectural choices, including filter sizes, network depth, width, and the selection of activation functions.
- The regularisation strategy must be documented and justified, with specific reference to dropout rates and batch normalisation placement.
- The choice of optimiser and learning rate schedule must be justified with reference to relevant literature or empirical observations.
- A minimum of two distinct hyperparameter configurations (e.g., varying learning rate or batch size) must be trained and evaluated. Validation loss and accuracy curves must be reported for each configuration.

4.3 Model B — Transfer Learning

Students shall employ at least one pretrained backbone architecture (e.g., ResNet-50, EfficientNet-B0, or MobileNetV2), replacing and retraining the top classification layers. The following three transfer learning strategies must be implemented and compared:

- **Feature extraction:** The backbone is entirely frozen; only the newly added classification head is trained.
- **Partial fine-tuning:** The uppermost blocks of the backbone are unfrozen and trained jointly with the classification head.
- **Full fine-tuning:** The entire backbone is unfrozen and trained end-to-end using a suitably low learning rate.

Validation metrics must be reported for all three strategies, and students are expected to discuss the observed trade-offs in terms of accuracy, computational cost, and generalisation.

4.4 Out-of-Distribution (OOD) Detection

Students must implement a baseline out-of-distribution detection mechanism capable of flagging input images as belonging to an unknown or unseen category. A threshold-based approach is considered an acceptable baseline. The following are required:

- A description and justification of the selected OOD detection rule.
- Construction of a held-out evaluation subset containing novel or previously unseen categories. This may be simulated by withholding a subset of classes from the training procedure.
- A report of precision and recall metrics for the detection of known versus unknown instances on the held-out subset.

4.5 Evaluation

The following evaluation outputs are required for all principal experiments:

- Per-class precision, recall, and F1 score.
- Macro-averaged F1 score and overall classification accuracy.
- A confusion matrix presented as a labelled heatmap.

- Training and validation loss and accuracy curves for all reported experimental configurations.

4.6 Reproducibility

All submitted work must be fully reproducible. The following conditions must be satisfied:

- A single self-contained Jupyter Notebook (.ipynb) must be provided, capable of executing the complete pipeline — including data loading, a training run of limited epochs, and full evaluation — without modification.
- A dependency specification file (requirements.txt or environment.yml) must be included.
- Random number generators must be seeded consistently throughout the codebase (Python, NumPy, PyTorch/TensorFlow) to ensure deterministic and reproducible results.

5. Deliverables

The following items must be submitted:

- **Jupyter Notebook (.ipynb):** A fully annotated notebook containing all code, with concise inline commentary.
- **Technical Report (PDF, 2–3 pages):** A written report summarising the methodology, principal findings, and a critical discussion of results.
- **Optional:** A Colab link, saved model weights, or a sample submission file may be included as supplementary material.